Week 5 Report

Jenkins, CI/CD & GitHub Integration
DevOps Internship | Davine Technologies

1. CI/CD

Continuous Integration (CI) means developers merge code frequently and every change is automatically built and tested, so problems are found early. **Continuous Delivery/Deployment (CD)** extends this by automatically preparing, and optionally releasing, the tested build to an environment. Together they make software delivery faster, safer and repeatable.

```
Commit --> Build --> Test --> Validate/Scan --> Package --> Deploy --> Monitor
```

2. Jenkins Architecture

Jenkins is an open-source automation server written in Java. A **controller** (master) schedules jobs, stores configuration and shows the UI; **agents** (nodes) execute the builds, so work can be spread across machines. Functionality is extended with **plugins** (Git, GitHub, Pipeline, Docker). Each run of a job is a **build** with its own number and Console Output.

Component	Role
Controller	Manages jobs, scheduling, UI and configuration
Agent / Node	Machine that runs the build steps
Executor	Slot on an agent that runs one build at a time
Plugin	Adds integrations and features
Workspace	Directory where a build checks out and builds code

3. Freestyle Jobs

A **Freestyle project** is configured through the Jenkins web form: source code (Git URL and branch), build triggers, build steps (Execute shell) and post-build actions. It is quick for simple jobs but the configuration is stored in Jenkins, not in the repository.

4. Declarative Pipelines and the Jenkinsfile

A **Declarative Pipeline** describes the whole CI/CD process as code in a **Jenkinsfile** stored in the repository (Pipeline as Code). It is versioned, reviewable and reproducible. The structure is pipeline { agent, environment, stages { stage { steps } }, post }.

```
pipeline {
  agent any
  triggers { githubPush() }
  stages {
    stage('Checkout')   { steps { checkout scm } }
    stage('Build')      { steps { sh 'npm run build' } }
    stage('Test')       { steps { sh 'npm test' } }
    stage('Validation') { steps { sh './scripts/validate.sh' } }
  }
  post {
    success { echo 'SUCCESS' }
    failure { echo 'FAILURE: check Console Output' }
  }
}
```

5. Pipeline Stages

Stage	Purpose
Checkout	Fetch the source code from GitHub
Build	Compile or check the code (npm run build)
Test	Run automated tests; fail the pipeline on errors
Validation	Extra checks: required files, secret scan, script syntax, dependency audit

If any stage fails, later stages are skipped and the post { failure } block runs, so broken code never moves forward.

6. GitHub Webhooks

A **webhook** is an HTTP callback: when a developer pushes code, GitHub sends a POST request to Jenkins at ***/github-webhook/*** and Jenkins starts the job immediately. This is faster and cheaper than polling the repository on a schedule. Jenkins must be reachable from GitHub (a public URL or a tunnel such as ngrok for local setups).

```
Developer --git push--> GitHub --POST /github-webhook/--> Jenkins --> Pipeline runs
```

7. Jenkins Credentials

Jenkins stores secrets (GitHub tokens, Docker Hub passwords, SSH keys) in an encrypted **credential store** and references them by ID. They are masked in logs and never written into the Jenkinsfile or repository. Use a GitHub Personal Access Token rather than a password.

```
withCredentials([usernamePassword(credentialsId: 'github-pat',
    usernameVariable: 'GH_USER', passwordVariable: 'GH_TOKEN')]) {
  sh 'git ls-remote https://$GH_USER:$GH_TOKEN@github.com/user/repo.git'
}
```

8. Hands-on Activity: Failure, Troubleshooting, Resolution

A webhook-triggered pipeline with four stages (Checkout, Build, Test, Validation) was created for a small Node.js application. A controlled error was introduced by changing a unit test's expected value from 5 to 6.

Phase	What happened
Failure	Test stage failed; Validation was skipped; build marked FAILURE
Troubleshooting	Console Output showed 'not ok 1 - add() adds two numbers', expected: 6, actual: 5. The failure was reproduced locally with npm test
Root cause	Wrong expected value in the test (application code was correct)
Resolution	Corrected the test, pushed; the webhook started a new build and all stages passed

9. CI/CD Best Practices

Practice	Why
Keep the Jenkinsfile in the repository	Versioned, reviewable pipeline
Fail fast and stop on the first failing stage	Broken code never reaches later stages
Read the Console Output before re-running	Fix the root cause, not the symptom
Store secrets in Jenkins Credentials	Prevents leaks in code and logs
Use webhooks instead of polling	Instant feedback, less load
Keep builds fast and automated	Developers get quick feedback

10. Conclusion

Week 5 showed how automation takes code from commit to validated build. A Jenkins pipeline triggered by GitHub webhooks tests every change, fails safely and gives clear feedback, which is the base for the Docker, Kubernetes and deployment pipelines in the coming weeks.